FINAL-YEAR B.TECH PROJECT PROPOSAL

Context Rot Firewall

Real-Time Context Health Management for Long-Running AI Agents

1. Executive Summary

Programme	B.Tech Computer Science & Engineering
Project type	Final-Year Major Project
Target duration	2 semesters (approx. 6 months of active build time)
Domain	AI Systems Engineering / Agentic AI Infrastructure
Date	July 2026

As AI agents move from single-turn chat to long-running autonomous workflows — coding agents, research agents, multi-hour computer-use sessions — a well-documented failure mode has emerged: **context rot**, the measurable degradation of model output quality as the context window fills with accumulated, increasingly irrelevant tokens. This is not context overflow (hitting the token limit); it happens well before the limit, simply because the model's attention gets diluted across noise.

I propose to build the **Context Rot Firewall**: a middleware layer that sits between an autonomous agent and its context window, continuously scoring every message and tool output for relevance to the agent's current sub-goal, and actively compressing, archiving, or evicting low-relevance content before it degrades reasoning — instead of the naive "keep everything" or "truncate oldest-first" strategies most agent frameworks use today.

> **Core deliverable:** a pluggable Python library + dashboard that wraps around any agent loop, and a benchmark demonstrating that task success rate holds up over long sessions with the firewall active, versus measurably degrading without it.

The project is engineering-heavy and ML-light — no model training or fine-tuning is required, which keeps it feasible within a student budget and timeline. It sits inside an active, fast-moving subfield of AI (context engineering for agents) with real, citable industry research behind the problem statement, and I believe there is a genuine gap in the existing tooling landscape, detailed in Section 5.

2. Problem Statement

2.1 What context rot actually is

Context rot is the term used across recent industry and academic work for a specific, measurable phenomenon: as more tokens are added to an LLM's input, output quality decreases — even when the model is nowhere near its advertised context limit. Chroma's widely-cited 2025 research tested 18 frontier models and found the effect present in every single one, not just some.

For agentic systems specifically, this is compounding rather than incidental. Every tool call, file read, search result, and intermediate reasoning step adds tokens to the running context. None of it gets removed by default in most agent frameworks, so the signal-to-noise ratio inside the context steadily degrades across a session, and the model's ability to attend to what actually matters — the current sub-goal — drops accordingly. Practitioner write-ups from 2026 report that agent coherence visibly declines once sessions pass roughly 20–30 turns, well before any token ceiling is hit.

The underlying mechanism is architectural, not a model bug: attention weights are normalized (softmax) across the entire context, so as context length grows, each individual token — including early, important instructions — receives a proportionally smaller share of attention. Separately, models exhibit a documented "lost-in-the-middle" positional bias, performing best when relevant information sits at the very start or end of the context and worse when it is buried in the middle.

2.2 Why this matters right now

- **Timing:** 2026 is the year long-horizon agents (coding agents, browser/computer-use agents, multi-step research agents) are moving from demo to production, which is exactly when this failure mode becomes operationally important rather than academic.
- **No standard fix:** most agent frameworks still handle this with blunt strategies — fixed-size sliding windows, oldest-first truncation, or leaving it entirely to the base model — rather than an active, relevance-aware management layer.
- **Safety angle:** degraded attention doesn't only blunt task performance. Recent work has shown that as padding/context length grows, safety-relevant instructions defined early in a session can also get diluted — a secondary but relevant motivation, even though this project's core contribution is systems engineering, not classification.

3. Proposed Solution

3.1 Concept

I propose a middleware layer — not a new agent framework — that intercepts the context an agent is about to send to the model on every turn, and actively manages what stays, what gets compressed, and what gets moved to retrievable cold storage. The

guiding principle: *relevance to the current sub-goal*, not recency and not raw token budget, determines what the model actually sees.

3.2 System architecture

Component	Function
Context Interceptor	Wraps the agent's LLM call; captures the full running message/tool-output history before each generation step.
Relevance Scorer	Scores each chunk in the running context against the current sub-goal. Two-tier: (1) cheap embedding-similarity pass for bulk scoring, (2) LLM-judged relevance pass only for borderline chunks, to keep cost down.
Compression Engine	Chunks below the relevance threshold get summarized into a short note rather than deleted outright — preserving gist while freeing tokens.
Cold Storage / Retrieval	Full original chunks are archived (vector store) and remain retrievable if a later sub-goal needs them back — analogous to OS memory paging.
Context Health Dashboard	Live view of token budget usage, relevance-score distribution, eviction/compression events, and (in eval mode) task-success-rate over session length.

3.3 How it differs from "just summarizing everything"

Naive summarization-on-overflow (what most agent harnesses do today, including basic `/compact`-style commands) compresses everything indiscriminately once a token ceiling is approached. The Context Rot Firewall instead scores continuously and selectively, so highly relevant recent tool output is kept verbatim while stale, irrelevant material is compressed early — before it ever contributes to attention dilution, and independent of whether the token ceiling is close.

4. Feasibility Study

4.1 Technical feasibility

Requirement	Feasibility
Compute	Low Runs on free-tier Colab/Kaggle T4, or even CPU for the scoring layer. No fine-tuning; only inference calls (embeddings + occasional relevance-judgment LLM calls) and classic ML (thresholding, not training a network).
Models needed	Low Open-weight small model for the agent loop itself (e.g. Llama-3.1-8B-Instruct or Qwen2.5-7B, 4-bit quantized) plus a small embedding model (e.g. `all-MiniLM-L6-v2`, free, CPU-friendly).
Data	Low No large labeled dataset required. Evaluation uses existing long-horizon agent benchmarks (see 4.3) plus self-generated multi-step tasks (e.g. multi-file coding tasks, multi-step research tasks).
Engineering complexity	Medium The core engine (interceptor + scorer + compressor + cold storage) is standard software engineering: async wrappers, a vector DB (Chroma/FAISS, free), and a scoring/eviction algorithm. No novel ML research required to reach a working v1.
Integration surface	Medium Wrapping an existing agent loop (LangChain agent, or a raw tool-calling loop against an API) is well documented; Playwright-based browser agents add moderate but manageable complexity for Sem 2's generalization demo.

4.2 Time & skill feasibility

Every component draws on standard, well-documented skills: Python, classifier/threshold-style logic, and dashboarding (Streamlit). The primary new skill area is working with vector databases and agent tool-calling loops, both of which have shallow learning curves with strong free documentation and are increasingly standard in AI-systems engineering.

4.3 Evaluation feasibility

A benchmark does not need to be built from scratch. The evaluation methodology can adapt Chroma's Context Rot approach (an extended Needle-in-a-Haystack test with semantic matching, plus a conversational QA evaluation called LongMemEval) to measure task accuracy at increasing context lengths, with and without the firewall active. A second, cheaper evaluation track uses self-authored multi-step coding or research tasks run to 40–60 turns, tracking whether the agent stays on-goal.

4.4 Cost feasibility

Entirely buildable at zero monetary cost: open-weight models on free-tier compute, free/open vector databases, and API-based evaluation limited to a small number of calls against a free-tier or low-cost hosted model for the final comparison runs. No paid dataset, no paid compute is required for a complete, demoable Sem 1 + Sem 2 deliverable.

5. Competitive & Related-Work Landscape

This is an active space with real existing tools — it is important to be upfront about that, and to be precise about where the genuine gap sits.

Existing tool / work	What it does	Where the gap remains
Letta (formerly MemGPT)	OS-inspired agent runtime with tiered memory (core / recall / archival); the agent itself decides what to page in and out via tool calls, self-editing its own memory.	Full agent runtime, not a lightweight add-on — adopting it means agents must run *inside* Letta, with real framework lock-in. Memory curation is left to the agent's own judgement, which is inconsistent; there is no dedicated, continuous relevance-scoring layer independent of the agent's self-assessment.
Mem0	A bolt-on memory layer (extract → consolidate → retrieve loop) usable with any existing agent stack, focused on long-term personalization/fact recall across sessions.	Optimized for cross-session personalization recall, not within-session, turn-by-turn context health during a single long-running task. Doesn't score/evict tool-output noise as it accumulates mid-task.
Hindsight	Passive memory extraction — automatically decides what to remember from agent interactions without agent-side deliberation.	Same focus as Mem0: durable memory across sessions, not live in-session context triage during a single long agentic run.
Claude Code-style `/compact` and similar summarize-on-overflow commands	Compress the whole context once a token ceiling is approached.	Reactive and indiscriminate — triggers on token count, not relevance; compresses everything at once rather than continuously evicting irrelevant chunks early, before they dilute attention.
Chroma's "Context Rot" research	Rigorously documents and measures the phenomenon across 18 frontier models; the primary evidence base motivating this project.	A research paper/benchmark, not a deployable tool. It diagnoses the disease; it does not ship a treatment.

> **The gap this project targets:** nearly all existing tools optimize for *persistent, cross-session* memory (what should the agent remember tomorrow). Very little tooling exists for *live, within-session context triage* — continuously and automatically deciding, turn by turn, what the model should keep paying attention to *right now*, independent of the agent's own self-assessment, and without requiring adoption of an entire new agent runtime. That is a genuinely narrower, more buildable, and currently underserved slice of the problem.

6. Why I Believe This Is a Strong Final-Year Project

- **Not a research replication.** This is not a reproduction of a known result; it is a scoped tool occupying real, currently-thin space between two categories of existing work (persistent memory frameworks vs. reactive compaction commands).
- **Demoable, not abstract.** The core result is a graph: task success rate over session length, with vs. without the firewall. That is a clean, convincing, live demo — not a paper full of numbers nobody outside the field can evaluate.
- **Infrastructure framing.** "I built a pluggable middleware layer that any agent framework can wrap" is a stronger interview sentence than "I built a classifier," because it signals systems thinking, not just an ML exercise.
- **Low-risk build profile.** No training, no large datasets, no paid infrastructure — the project's risk is entirely in engineering execution, which is controllable, not in whether an experimental technique works at all.

7. Roadmap

7.1 Semester 1 — Core Engine (Months 1–3)

Month	Milestones
Month 1	• Set up a baseline agent loop (raw tool-calling loop against an open-weight model, or a minimal LangChain agent) with no context management — this serves as the control/baseline. • Stand up embedding pipeline (`sentence-transformers`) and a vector store (Chroma DB) for archival storage. • Define the relevance-scoring interface: given (current sub-goal, context chunk) → relevance score.
Month	• Build the Context Interceptor: wrap the agent's LLM call so every context chunk is visible to the firewall engine before each

2	generation step. • Implement the two-tier scorer: embedding-similarity pass (bulk) + LLM-judged pass (borderline chunks only, to control cost). • Implement the Compression Engine: below-threshold chunks → short summary + full original archived to cold storage.
Month 3	• Implement retrieval-back-from-cold-storage when a later sub-goal needs an evicted chunk. • Build first version of the Context Health Dashboard (Streamlit): live token budget, relevance distribution, eviction log. • Run first controlled comparison: same multi-step task, with vs. without the firewall, on a small task set. Produce the first "does it work at all" result.

7.2 Semester 2 — Evaluation, Generalization & Polish (Months 4–6)

Month	Milestones
Month 4	• Build the proper benchmark suite: adapt Chroma-style Needle-in-a-Haystack + semantic matching tasks, plus 3–5 self-authored long multi-step tasks (coding + research style). • Run full degradation benchmark: task success rate vs. session length/turn count, with and without the firewall, across at least two task types.
Month 5	• Generalize the wrapper to a second agent type (e.g. a Playwright-driven browser agent) to demonstrate the engine is not hardcoded to one framework. • Tune the relevance-scoring threshold and compression strategy based on Month 4 results; add configurable policies (aggressive vs. conservative eviction).
Month 6	• Finalize the Context Health Dashboard with the full comparison results built in. • Write up the project report: problem statement, architecture, benchmark methodology, results, and the competitive-gap framing from Section 5. • Prepare the live demo script (baseline agent visibly degrading vs. firewall-protected agent staying on-task, on stage).

8. Risk Assessment

Risk	Likelihood / Impact	Mitigation
Relevance scorer is inaccurate, evicts things that actually mattered	**Medium / Medium**	Keep evicted content in retrievable cold storage rather than deleting outright, so a wrong eviction is recoverable, not fatal; tune threshold against benchmark results rather than guessing.
LLM-judged relevance pass adds too much latency/cost at scale	**Medium / Low**	Two-tier design keeps the expensive LLM-judgment call only for borderline chunks after the cheap embedding pass filters the bulk — this is a designed-in mitigation, not an afterthought.
Benchmark results are noisy / hard to attribute cleanly to the firewall	**Low / Medium**	Use a fixed task set with deterministic scoring where possible (e.g. code tasks with pass/fail unit tests) rather than subjective quality judgments, and report the well-established Chroma-style methodology alongside the self-authored tasks for credibility.
External commitments or scheduling constraints compress available build time in a given semester	**Medium / Medium**	Roadmap is intentionally modular — Semester 1 alone (core engine + first comparison result) is a complete, gradeable deliverable on its own if Semester 2 generalization work gets compressed.

9. Tech Stack Summary

Layer	Tools
Agent model	Llama-3.1-8B-Instruct or Qwen2.5-7B-Instruct (4-bit quantized), HuggingFace Transformers
Embeddings	sentence-transformers (`all-MiniLM-L6-v2`)
Vector / cold storage	ChromaDB (local, free)
Agent orchestration	Raw tool-calling loop or LangChain; Playwright for the Sem 2 browser-agent generalization demo
Dashboard	Streamlit
Compute	Google Colab / Kaggle free-tier T4 GPU, or CPU-only where feasible

10. Deliverables Checklist

- Working Context Rot Firewall library (pip-installable local package) with documented API for wrapping an arbitrary agent loop
- Context Health Dashboard (Streamlit) showing live token usage, relevance scores, and eviction history
- Benchmark suite + results: task success rate vs. session length, with/without the firewall, across at least two agent types
- Written project report including the competitive-landscape framing (Section 5) as the novelty argument
- Live demo script for evaluation day

11. Conclusion

I am confident this project is achievable within the proposed two-semester timeline, requires no paid infrastructure or large-scale data collection, and addresses a real, currently underserved gap between existing agent-memory frameworks and reactive context-compaction tools. I request approval to proceed with the Context Rot Firewall as my final-year major project on this basis.

12. References & Further Reading

- Chroma – "Context Rot" research (foundational benchmark for the problem statement): morphllm.com/context-rot ; understandingai.org/p/context-rot-the-emerging-challenge
- Context Engineering: Agent Reliability Playbook 2026 – digitalapplied.com/blog/context-engineering-agent-reliability-playbook-2026
- TechAhead – "The Context Rot Problem: Why Your AI Agents Forget Faster Than They Learn" – techaheadcorp.com/blog/context-rot-problem
- Redis – "Context rot explained (& how to prevent it)" – redis.io/blog/context-rot
- Liu et al., "Lost in the Middle" (positional bias in long-context LLMs), 2024
- Packer et al., "MemGPT: Towards LLMs as Operating Systems," 2023 – original virtual-context-management paper behind Letta
- Letta vs. Mem0 vs. Hindsight comparisons (competitive landscape source) – vectorize.io/articles/mem0-vs-letta ; vectorize.io/articles/hindsight-vs-letta ; tokenmix.ai/blog/ai-agent-memory-mem0-vs-letta-vs-memgpt-2026
- arXiv 2604.02688 – context management practices in long-running code-first LLM agents
- arXiv 2512.02445 – "When Refusals Fail: Unstable Safety Mechanisms in Long-Context LLM Agents"

This proposal was compiled with current (July 2026) web research on the state of context-rot literature and agent-memory tooling; verify exact tool feature sets against current documentation before submission, since this is a fast-moving space.